Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа №3

по курсу «Моделирование»

«Построение стохастической модели в заданной предметной области»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Домрачева А.Б.

Москва 2026

1 Постановка задачи

Целью данной лабораторной работы является кластеризация объектов по входным данным, представленным в файле diabetes.csv.

Необходимо выполнить следующее:

1. Выбор бинаризации показателей.
2. Выбор коэффициента взаимосвязи показателей.
3. Сокращение размерности задачи при наличии зависимых признаков.
4. Кластеризация данных по методу k-means.
5. Интерпретация полученных классов.

Ссылка на набор данных: `https://drive.google.com/drive/folders/1ziAhYiu85_K2Jjls3zxLayfpvvW61Gxf?usp=drive_link`.

2 Построение модели

На первом этапе числовые признаки переводятся в бинарную форму. Рассматривались два способа задания порога:

$$z_i = \begin{cases} 1, & x_i > \overline{x}, \\ 0, & x_i \leq \overline{x}, \end{cases} \qquad \overline{x} = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

$$z_i = \begin{cases} 1, & x_i > M(x), \\ 0, & x_i \leq M(x). \end{cases}$$

Здесь x_i — значение признака для i-го объекта, z_i — бинаризованное значение, $\overline{x}$ — среднее значение признака по выборке, $M(x)$ — медиана признака, n — число объектов.

В результате в работе принята бинаризация по медиане. Для каждого объекта значение признака заменяется логическим показателем «выше медианы / не выше медианы». В рассматриваемом наборе данных использованы следующие медианные пороги:

Признак	Медиана	Признак	Медиана
Pregnancies	3	Insulin	30.5
Glucose	117	BMI	32
BloodPressure	72	DiabetesPedigreeFunction	0.3725
SkinThickness	23	Age	29

После бинаризации для каждой пары признаков строится таблица сопряженности

	$B = 1$	$B = 0$
$A = 1$	a	b
$A = 0$	c	d

где a — число объектов, у которых оба признака выше порога, b — число объектов, у которых первый признак выше порога, а второй нет, c — число объектов, у которых второй признак выше порога, а первый нет, d — число объектов, у которых оба признака не превышают порог.

После бинаризации были построены таблицы с коэффициентами ассоциации, Бернштейна, Пирсона и Юла. В итоге был выбран коэффициент ассоциации. Его выбор обусловлен тем, что он является симметричным, принимает значения в диапазоне от -1 до 1, сохраняет знак зависимости и позволяет уверенно отделять сильные и слабые связи между бинаризованными признаками.

Коэффициент ассоциации

$$K_{\text{ассоц}} = \frac{ad - bc}{ad + bc}.$$

Здесь a, b, c, d — элементы таблицы сопряженности. Коэффициент принимает значения от -1 до 1, сохраняет знак зависимости и симметрично оценивает совместные появления высоких и низких уровней признаков.

Матрица коэффициентов ассоциации приведена на рисунке 1.

Матрица коэффициентов ассоциации

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
Pregnancies	1.0000	0.2401	0.3456	-0.0557	-0.4127	0.0935	-0.0526	0.8733
Glucose	0.2401	1.0000	0.3526	0.1644	0.1297	0.3386	0.1194	0.4033
BloodPressure	0.3456	0.3526	1.0000	0.1806	-0.0995	0.3690	-0.0053	0.4909
SkinThickness	-0.0557	0.1644	0.1806	1.0000	0.6754	0.7075	0.2215	0.0028
Insulin	-0.4127	0.1297	-0.0995	0.6754	1.0000	0.2162	0.3529	-0.3532
BMI	0.0935	0.3386	0.3690	0.7075	0.2162	1.0000	0.1451	0.1551
DiabetesPedigreeFunction	-0.0526	0.1194	-0.0053	0.2215	0.3529	0.1451	1.0000	0.0521
Age	0.8733	0.4033	0.4909	0.0028	-0.3532	0.1551	0.0521	1.0000

Рис. 1 — Матрица коэффициентов ассоциации

По матрице коэффициентов ассоциации наиболее сильные связи имеют пары Pregnancies–Age (0.8733), SkinThickness–BMI (0.7075), SkinThickness–Insulin (0.6754), BloodPressure–Age (0.4909). Эти значения прямо указывают на дублирование информации внутри нескольких групп признаков и позволяют перейти к сокращению размерности.

Сокращение размерности выполняется по следующему правилу. Строится матрица модулей коэффициентов связи, диагональные элементы зануляются, после чего признаки объединяются в компоненты связности при пороге сильной связи $|r| \geq 0.45$. В каждой компоненте в качестве представителя сохраняется признак с максимальной суммарной связью внутри своей группы. По рассчитанной матрице коэффициентов были получены две зависимые группы:

- Pregnancies, BloodPressure, Age, из которых сохраняется Age;
- SkinThickness, Insulin, BMI, из которых сохраняется SkinThickness.

Признаки Glucose и DiabetesPedigreeFunction не вошли в группы сильной зависимости и были оставлены без изменений. В результате итоговое пространство признаков для кластеризации имеет четыре координаты: Glucose, SkinThickness, DiabetesPedigreeFunction, Age.

После отбора координат каждый признак стандартизуется, чтобы привести шкалы измерения к сопоставимому виду. Затем на стандартизованных данных

выполняется кластеризация методом k-means при $k = 2$. Для графического представления полученного разделения дополнительно применяется проекция главных компонент в двумерное пространство.

Схема модели представлена на рисунке 2.

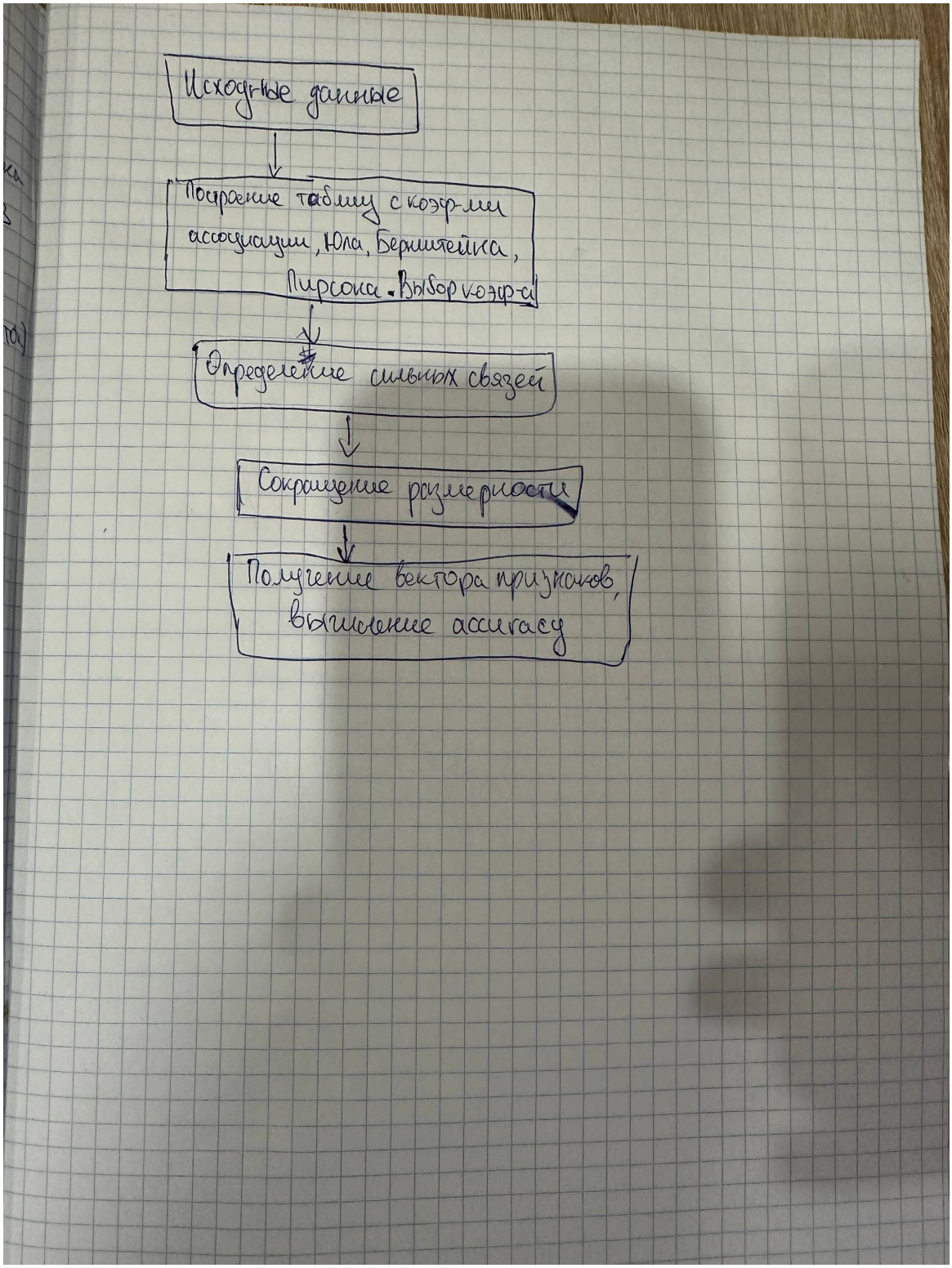

Рис. 2 — Схема модели

3 Решение вычислительной задачи

Метод k-means разбивает множество объектов на k кластеров так, чтобы минимизировать суммарное расстояние объектов до центров своих кластеров:

$$J = \sum_{j=1}^{k} \sum_{x_i \in S_j} \|x_i - \mu_j\|^2.$$

Здесь J — функционал качества разбиения, k — число кластеров, x_i — вектор признаков i-го объекта, S_j — множество объектов j-го кластера, μ_j — центр j-го кластера. Работа метода k-means может быть представлена следующими шагами.

1. Задаётся число кластеров k.
2. В пространстве признаков выбираются начальные центры кластеров $\mu_1, \mu_2, \ldots, \mu_k$.
3. Для каждого объекта x_i вычисляются расстояния до всех центров кластеров.
4. Объект x_i относится к тому кластеру, центр которого находится ближе всего.
5. Для каждого кластера пересчитывается новый центр μ_j как среднее арифметическое векторов объектов, попавших в этот кластер.
6. Шаги 3–5 повторяются до тех пор, пока состав кластеров перестанет изменяться или пока изменение функционала J не станет пренебрежимо малым.

Таким образом, метод k-means поочередно уточняет принадлежность объектов к кластерам и положение центров кластеров, постепенно уменьшая внутрикластерную сумму квадратов.

Для оценки качества после согласования меток кластеров с целевой переменной Outcome используется коэффициент точности

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N}.$$

Здесь N_{correct} — число объектов, для которых согласованная метка кластера совпала с Outcome, N — общее число объектов выборки.

4 Анализ модели

Результат кластеризации в двумерной проекции приведён на рисунке 3. Изображение отражает не исходное четырёхмерное пространство, а его двумерное представление, используемое только для визуального анализа.

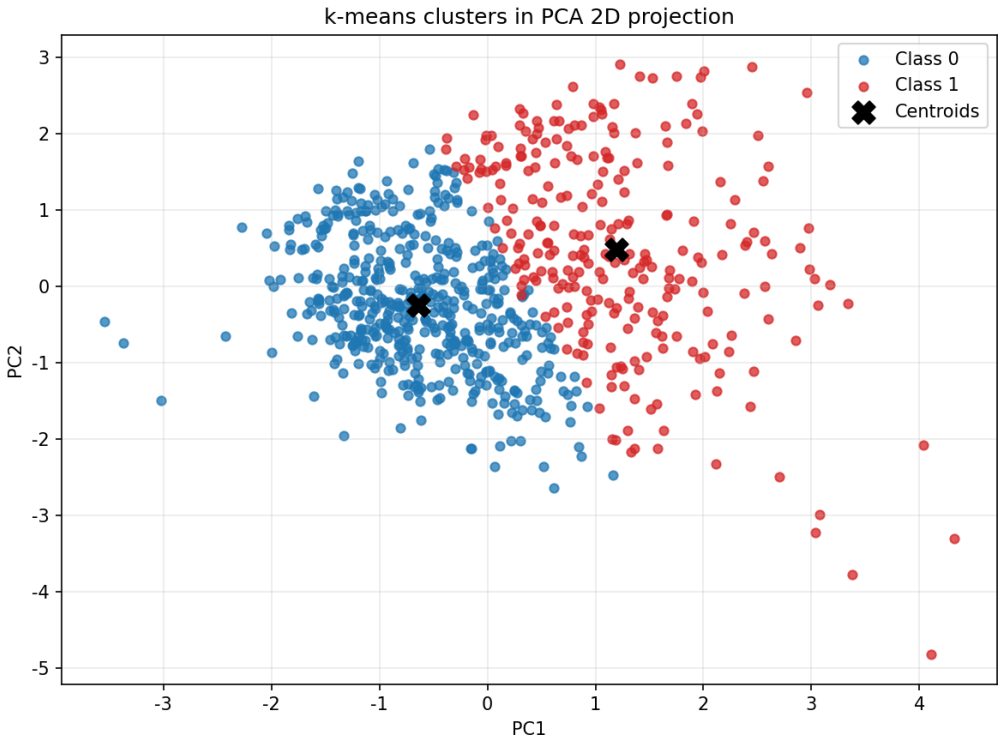

Рис. 3 — Кластеры объектов в двумерной проекции

По результатам сокращения размерности для кластеризации были отобраны признаки Glucose, SkinThickness, DiabetesPedigreeFunction, Age. После работы алгоритма получены два кластера: класс 0 содержит 501 объект, класс 1 содержит 267 объектов. Точность согласования меток кластеров с Outcome составляет 0.74349, что соответствует 571 верному совпадению из 768 объектов. Для сравнения, при использовании всех восьми исходных признаков точность составляет 0.67578, то есть 519 верных совпадений из 768. Следовательно, сокращение размерности в данной задаче улучшает качество итогового разбиения.

Интерпретация классов выполнена по значениям центроидов относительно медиан исходных признаков.

Таблица 1: Характеристики центроидов полученных кластеров

Класс	Число объектов	Glucose	SkinThickness	DiabetesPedigreeFunction	Age
0	501	106.511	21.275	0.416	27.471
1	267	147.884	19.150	0.576	44.067

Сопоставление полученных классов с Outcome даёт следующую матрицу согласования:

Таблица 2: Матрица согласования кластеров и целевой переменной Outcome

Кластер	Outcome = 0	Outcome = 1
0	402	99
1	98	169

По рисунку 3 видно, что кластеры разделяются удовлетворительно, однако между ними сохраняется зона пересечения. После сокращения размерности итоговое разбиение строится по признакам Glucose, SkinThickness, DiabetesPedigreeFunction и Age, причем полученные центроиды различаются по всем этим координатам.

5 Вывод

В ходе лабораторной работы была построена стохастическая модель кластеризации объектов по данным diabetes.csv. Была выполнена бинаризация признаков по медианным порогам, проведён анализ коэффициентов связи и для дальнейшей работы выбран коэффициент ассоциации. На его основе выполнено сокращение размерности до четырёх признаков Glucose, SkinThickness, DiabetesPedigreeFunction и Age. На отобранных координатах метод k-means сформировал два кластера объектов, а точность согласования с Outcome составила 0.74349.

6 Практическая реализация

Листинг 1. Исходный код программы.

```
import csv
import random

import matplotlib
import numpy as np

matplotlib.use("Agg")
import matplotlib.pyplot as plt

plt.rcParams["font.family"] = "DejaVu Sans"
plt.rcParams["axes.unicode_minus"] = False

ASSOCIATION_MATRIX_FILE = "association_matrix.csv"
STRONG_RELATION_THRESHOLD = 0.45


def load_data(path):
    with open(path, "r", encoding="utf-8", newline="") as file:
        reader = csv.DictReader(file)
        rows = list(reader)
    feature_names = [name for name in reader.fieldnames if name != "
   Outcome"]
    data = np.array([[float(row[name]) for name in feature_names]
   for row in rows], dtype=float)
    outcome = np.array([int(row["Outcome"]) for row in rows], dtype=
   int)
    return feature_names, data, outcome


def median_binarize(data):
    medians = np.median(data, axis=0)
    return medians, data > medians


def load_matrix_csv(path, expected_feature_names):
    with open(path, "r", encoding="utf-8", newline="") as file:
        reader = csv.reader(file)
        header = next(reader)
        file_features = header[1:]
        if file_features != expected_feature_names:
            raise ValueError("association_matrix.csv does not match
   feature order in diabetes.csv")
        rows = list(reader)
    row_features = [row[0] for row in rows]
    if row_features != expected_feature_names:
```

```
        raise ValueError("association_matrix.csv row labels do not
    match diabetes.csv")
    matrix = np.array([[float(x) for x in row[1:]] for row in rows],
    dtype=float)
    return matrix


def strength_matrix(matrix):
    strength = np.abs(matrix).astype(float)
    np.fill_diagonal(strength, 0.0)
    return strength


def connected_components(strength, threshold):
    count = strength.shape[0]
    adjacency = [[] for _ in range(count)]
    for i in range(count):
        for j in range(i + 1, count):
            if strength[i, j] >= threshold:
                adjacency[i].append(j)
                adjacency[j].append(i)
    visited = [False] * count
    components = []
    for start in range(count):
        if visited[start]:
            continue
        stack = [start]
        visited[start] = True
        component = []
        while stack:
            node = stack.pop()
            component.append(node)
            for neighbor in adjacency[node]:
                if not visited[neighbor]:
                    visited[neighbor] = True
                    stack.append(neighbor)
        components.append(sorted(component))
    return components


def select_features(strength, threshold, feature_names):
    components = connected_components(strength, threshold)
    selected = []
    for component in components:
        if len(component) == 1:
            selected.append(component[0])
```

```
            continue
        weights = []
        for idx in component:
            score = float(sum(strength[idx, other] for other in
    component if other != idx))
            weights.append((idx, score))
        representative = sorted(weights, key=lambda item: (-item[1],
     feature_names[item[0]]))[0][0]
        selected.append(representative)
    global_scores = strength.sum(axis=1)
    if len(selected) < 2:
        order = sorted(range(len(feature_names)), key=lambda idx: (-
    global_scores[idx], feature_names[idx]))
        for idx in order:
            if idx not in selected:
                selected.append(idx)
            if len(selected) >= 2:
                break
    return sorted(set(selected))


def standardize(points):
    mean = points.mean(axis=0)
    std = points.std(axis=0)
    std = np.where(std == 0, 1.0, std)
    standardized = (points - mean) / std
    return standardized


def kmeans_once(points, k, max_iterations, rng):
    indices = rng.sample(range(len(points)), k)
    centroids = points[np.array(indices, dtype=int)].astype(float)
    labels = None
    for _ in range(max_iterations):
        distances = ((points[:, None, :] - centroids[None, :, :]) **
     2).sum(axis=2)
        new_labels = np.argmin(distances, axis=1)
        if labels is not None and np.array_equal(new_labels, labels)
    :
            break
        labels = new_labels
        for cluster_id in range(k):
            mask = labels == cluster_id
            if np.any(mask):
                centroids[cluster_id] = points[mask].mean(axis=0)
            else:
```

```
                centroids[cluster_id] = points[rng.randrange(len(
points))]
    inertia = float(((points - centroids[labels]) ** 2).sum())
    return labels.astype(int), centroids.astype(float), inertia


def kmeans(points, k=2, n_init=30, max_iterations=300, seed=42):
    best = None
    for run in range(n_init):
        rng = random.Random(seed + run * 9973)
        labels, centroids, inertia = kmeans_once(points, k,
max_iterations, rng)
        if best is None or inertia < best["inertia"]:
            best = {"labels": labels, "centroids": centroids, "
inertia": inertia}
    return best


def align_clusters_to_outcome(labels, outcome):
    labels = np.asarray(labels, dtype=int)
    outcome = np.asarray(outcome, dtype=int)
    direct = float((labels == outcome).mean())
    flipped_labels = 1 - labels
    flipped = float((flipped_labels == outcome).mean())
    if flipped > direct:
        return flipped_labels, flipped
    return labels, direct


def pca_projection(points, dimensions=2):
    centered = points - points.mean(axis=0)
    _, _, vt = np.linalg.svd(centered, full_matrices=False)
    components = vt[:dimensions].T
    projection = centered @ components
    return projection, components, points.mean(axis=0)


def draw_cluster_projection(points_standardized, labels_aligned,
centroids_standardized, path):
    projection, components, mean = pca_projection(
points_standardized, dimensions=2)
    centroid_projection = (centroids_standardized - mean) @
components
    fig, axis = plt.subplots(figsize=(8, 6))
    colors = ["#1f77b4", "#d62728"]
    labels_aligned = np.asarray(labels_aligned, dtype=int)
```

```
    for cluster_id in [0, 1]:
        cluster_points = projection[labels_aligned == cluster_id]
        axis.scatter(
            cluster_points[:, 0],
            cluster_points[:, 1],
            s=24,
            alpha=0.75,
            color=colors[cluster_id],
            label=f"Class {cluster_id}",
        )
    axis.scatter(
        centroid_projection[:, 0],
        centroid_projection[:, 1],
        s=160,
        c="#000000",
        marker="X",
        label="Centroids",
    )
    axis.set_xlabel("PC1")
    axis.set_ylabel("PC2")
    axis.set_title("k-means clusters in PCA 2D projection")
    axis.grid(True, alpha=0.25)
    axis.legend()
    fig.tight_layout()
    fig.savefig(path, dpi=150)
    plt.close(fig)


def save_clusters_csv(data, feature_names, selected_indices,
    labels_raw, labels_aligned, outcome, path):
    selected_names = [feature_names[idx] for idx in selected_indices
    ]
    with open(path, "w", encoding="utf-8", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(["object_id"] + selected_names + ["
    cluster_raw", "cluster_aligned", "Outcome"])
        for i in range(len(data)):
            values = [f"{data[i, idx]:.6f}" for idx in
    selected_indices]
            writer.writerow([i + 1] + values + [int(labels_raw[i]),
    int(labels_aligned[i]), int(outcome[i])])


def save_cluster_summary_csv(feature_names, selected_indices,
    medians, data, labels_aligned, path):
```

```
    selected_names = [feature_names[idx] for idx in selected_indices
   ]
    header = ["class", "count"]
    for name in selected_names:
        header.append(f"{name}_centroid")
        header.append(f"{name}_vs_median")
    with open(path, "w", encoding="utf-8", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(header)
        for class_id in [0, 1]:
            mask = labels_aligned == class_id
            class_data = data[mask][:, selected_indices]
            if len(class_data):
                centroid = class_data.mean(axis=0)
            else:
                centroid = np.zeros(len(selected_indices), dtype=
   float)
            row = [class_id, int(mask.sum())]
            for local_idx, feature_idx in enumerate(selected_indices
   ):
                value = float(centroid[local_idx])
                direction = "above_median" if value > medians[
   feature_idx] else "below_median"
                row.extend([f"{value:.6f}", direction])
            writer.writerow(row)


def print_text_output(accuracy, labels_aligned, feature_names,
   selected_indices):
    labels_aligned = np.asarray(labels_aligned, dtype=int)
    count0 = int((labels_aligned == 0).sum())
    count1 = int((labels_aligned == 1).sum())
    selected = ", ".join(feature_names[idx] for idx in
   selected_indices)
    print(f"Selected features: {selected}")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Class 0 size: {count0}")
    print(f"Class 1 size: {count1}")


def main():
    feature_names, data, outcome = load_data("diabetes.csv")
    medians, _ = median_binarize(data)
    matrix = load_matrix_csv(ASSOCIATION_MATRIX_FILE, feature_names)
    strength = strength_matrix(matrix)
```

```
    selected_indices = select_features(strength,
   STRONG_RELATION_THRESHOLD, feature_names)
    selected_points = data[:, selected_indices]
    standardized_points = standardize(selected_points)
    km = kmeans(standardized_points, k=2, n_init=30, max_iterations
   =300, seed=42)
    labels_raw = km["labels"]
    labels_aligned, accuracy = align_clusters_to_outcome(labels_raw,
    outcome)

    best = {
        "selected_indices": selected_indices,
        "labels_raw": labels_raw,
        "labels_aligned": labels_aligned,
        "centroids_standardized": km["centroids"],
        "accuracy": accuracy,
    }

    save_clusters_csv(
        data,
        feature_names,
        best["selected_indices"],
        best["labels_raw"],
        best["labels_aligned"],
        outcome,
        "clusters.csv",
    )
    save_cluster_summary_csv(
        feature_names,
        best["selected_indices"],
        medians,
        data,
        best["labels_aligned"],
        "cluster_summary.csv",
    )
    draw_cluster_projection(
        standardized_points,
        best["labels_aligned"],
        best["centroids_standardized"],
        "kmeans_clusters_2d.png",
    )
    print_text_output(best["accuracy"], best["labels_aligned"],
   feature_names, best["selected_indices"])


if __name__ == "__main__":
```

```
    main()
```